



SQUAD: LegoDados - Projeto de Inteligência Legislativa & Engenharia de dados



Este repositório contém o Projeto Integrador da pós-graduação em Engenharia de Dados e Inteligência Artificial. O objetivo é desenvolver um pipeline de dados completo (ETL) que automatiza a captura, organização e análise de dados da API de Dados Abertos da Câmara dos Deputados.



Propósito do

Projeto:

Transformar o [oceano de dados brutos do legislativo brasileiro](#) em sinais acionáveis para consultorias de relações governamentais e empresas reguladas. O projeto visa substituir processos manuais e inconsistentes por uma arquitetura escalável que utiliza IA Generativa para classificação temática e resumos executivos.



Stack

Tecnológica:





Arquitetura e

Roadmap de Desenvolvimento:



O projeto está estruturado em cinco etapas principais. Abaixo está o status atual de desenvolvimento do que já foi mapeado e implementado:

1. **Exploração e Extração (Ingestão) [CONCLUÍDO]**
 - Desenvolvimento de scripts Python em `src/extraction.py` para consumo estruturado da API de Dados Abertos.
 - Extração modularizada através de extratores específicos: `DeputadosExtractor`, `PartidosExtractor`, `ProposicoesExtractor` e `VotacoesExtractor`.
 - Tratamento de paginação, resiliência contra erros de timeout e persistência do JSON bruto no diretório local `data_raw/` (Camada Bronze).

2. Diagnóstico e Configuração [CONCLUÍDO]

- Implementação de rotinas de validação inicial (src/diagnostico.py) disparadas antes da execução principal para garantir a integridade dos diretórios e conexões.
- Centralização das configurações e segurança através de variáveis de ambiente gerenciadas em src/config.py e .env.

3. Transformação e Carga (ETL) [CONCLUÍDO]

- Limpeza, padronização e processamento dos dados brutos utilizando Pandas (src/transformers.py).
- Modelagem relacional transformando arquivos JSON em estruturas adequadas para tabelas Fato e Dimensão (ex: fato_proposicoes_autores, fato_votacoes, fato_votos).
- Orquestração e execução da carga incremental (UPSERT por chave + delete/insert escopado nas associativas) em banco de dados PostgreSQL via SQLAlchemy (src/transformation.py), preservando `created_at` e os enriquecimentos de IA entre execuções.

4. Camada de Inteligência Artificial [CONCLUÍDO]

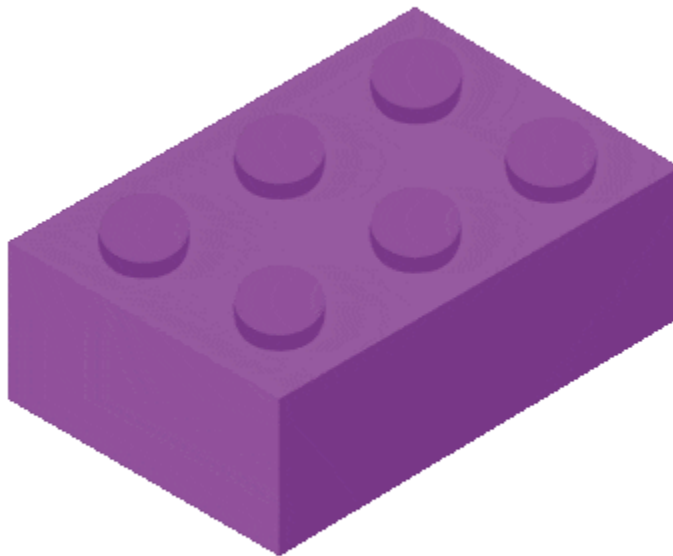
- **Caminho B — Resumo Executivo** (src/ai_layer.py, classe PipelineEtapa4): para cada proposição pendente, gera um resumo de até 3 frases em linguagem executiva via **GPT-4o-mini**, gravado em `fato_proposicoes.resumo_executivo / data_resumo`.
- **Caminho A — Classificação Temática por Embeddings** (src/classificacao_tematica.py, classe PipelineEtapa5): gera o embedding (**text-embedding-3-small**) da ementa de cada proposição e de um catálogo de ~11 temas de negócio (Tecnologia e IA, Tributário, Saúde, Trabalho e Previdência, Meio Ambiente, Economia e Finanças, Educação, Segurança Pública, Agronegócio, Infraestrutura e Transporte, Direitos e Cidadania), calcula a **similaridade de cosseno** entre eles e grava o tema de maior similaridade (ou "Outros" quando abaixo do limiar `LIMIAR_TEMA`) em `fato_proposicoes.tema / tema_score / data_tema`.
- **Modo de Simulação (Dry Run)**: ambas as etapas calculam e exibem a estimativa de custo (USD/BRL, tokens) antes de qualquer chamada real à API — controlado por `DRY_RUN` no .env.
- O `tema` classificado alimenta diretamente o digest e os alertas do workflow n8n da Etapa 5 — a IA não é decoração, ela muda o conteúdo do e-mail enviado.

5. Automação e Monitoramento [CONCLUÍDO]

- Workflow no **n8n** (n8n/bussola_publica_ingestao_diaria.json) agendado para rodar **diariamente às 06h** (cron `0 6 * * *`, timezone `America/Sao_Paulo`), executando `main.py/main2.py` via *Execute Command*.
- **Digest diário por e-mail** com as proposições mais relevantes das últimas 24h, já com **tema (embeddings)** e **resumo executivo (GPT)**.
- **Alerta de falha**: ramo dedicado que envia e-mail com o `stderr` caso o pipeline quebre.
- Guia completo de importação e configuração de credenciais em [n8n/GUIA_IMPORTACAO_n8n.md](#).

Modelo de Dados (DWH / Camada Relacional):

Para suportar as análises legislativas e o enriquecimento com Inteligência Artificial, os dados transformados foram estruturados em um modelo relacional (Fatos e Dimensões).



Tabelas de

Dimensão (Dim):

`dim_deputados`

- Armazena os dados cadastrais e identificadores únicos dos deputados federais.

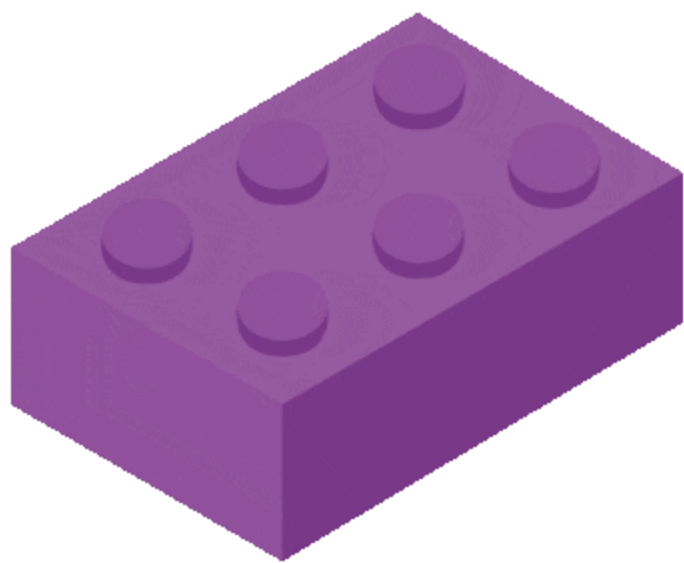
Campo	Tipo	Restrição	Descrição
id_deputado	int8	Primary Key	Identificador único do deputado na API da Câmara.
nome	text	Nullable	Nome parlamentar do deputado.
sigla_partido	text	Nullable	Sigla do partido político atual.
sigla_uf	text	Nullable	Estado (Unidade da Federação) pelo qual foi eleito.
id_legislatura	int8	Nullable	Identificador da legislatura atual.
url_foto	text	Nullable	Link para a foto oficial do parlamentar.
uri	text	Nullable	Link do endpoint oficial do deputado na API.

dim_partidos

- Dicionário de partidos políticos mapeados no pipeline.

Campo	Tipo	Restrição	Descrição
id_partido	int8	Primary Key	Identificador único do partido na API.
sigla	text	Nullable	Sigla oficial do partido político.
nome	text	Nullable	Nome completo do partido político.

Campo	Tipo	Restrição	Descrição
uri	text	Nullable	Link do endpoint oficial do partido na API.



Tabelas de Fato

(Fact):
fato_proposicoes

- Entidade central de análise que armazena os textos, metadados e os enriquecimentos de IA (resumo executivo + classificação temática).

Campo	Tipo	Restrição	Descrição
id_proposicao	int8	Primary Key	Identificador único da proposição (projeto de lei, PEC, etc).

Campo	Tipo	Restrição	Descrição
sigla_tipo	text	Nullable	Tipo da proposição (ex: PL, PEC, MPV).
numero	int8	Nullable	Número oficial da proposição no ano.
ano	int8	Nullable	Ano de apresentação da matéria legislativa.
ementa	text	Nullable	Texto original da ementa detalhando o objetivo do projeto.
data_apresentacao	timestampz	Nullable	Data e hora em que a matéria foi protocolada.
created_at	timestampz	Nullable	Data/hora da primeira ingestão do registro (não muda em re-execuções graças ao UPSERT).
resumo_executivo	text	[IA · Etapa 4]	Resumo analítico simplificado gerado via GPT-4o-mini (Caminho B).
data_resumo	timestampz	[IA · Etapa 4]	Timestamp de quando o resumo de IA foi gerado.
tema	text	[IA · Etapa 5]	Tema classificado por similaridade de cosseno entre embeddings (Caminho A). Alimenta o digest/alertas do n8n.

Campo	Tipo	Restrição	Descrição
tema_score	float8	[IA · Etapa 5]	Similaridade de cosseno (0–1) entre a ementa e o tema escolhido.
data_tema	timestampz	[IA · Etapa 5]	Timestamp de quando a classificação temática foi gerada.

Nota sobre `tema_score`: `classificacao_tematica.py` calcula e grava `tema`, `tema_score` e `data_tema` juntos no mesmo `UPDATE`. Na carga atual do banco, `tema` está preenchido para as proposições já classificadas, mas `tema_score` ficou `NULL` em parte das linhas — indício de execuções feitas com uma versão anterior do script (antes da gravação do score). Isso não bloqueia o uso de `tema` no digest/alertas do n8n; é um ponto de atenção para a próxima reexecução da classificação temática.

`fato_proposicoes_autores`

- Tabela associativa que mapeia a autoria ou coautoria de cada proposição legislativa.

Campo	Tipo	Restrição	Descrição
id_proposicao	int8	Nullable	ID da proposição (chave estrangeira para <code>fato_proposicoes</code>).
nome_autor	text	Nullable	Nome do parlamentar ou órgão autor da matéria.
tipo_autor	text	Nullable	Categoria do autor (ex: Deputado, Órgão Executivo).
uri_autor	text	Nullable	Link do endpoint do autor na API.

`fato_votacoes`

- Registra as sessões de votações ocorridas na Câmara para deliberação das matérias.

Campo	Tipo	Restrição	Descrição
id_votacao	text	Primary Key	Identificador alfanumérico único da votação.
descricao	text	Nullable	Detalhamento do que está sendo votado em plenário ou comissão.
data_hora_registro	timestampz	Nullable	Data e hora exata da sessão de votação.
aprovacao	int2	Nullable	Indicador binário/status se a matéria foi aprovada (1) ou não (0).
proposicao_objeto	text	Nullable	Descrição ou link da matéria que originou a votação.
created_at	timestampz	Nullable	Data/hora da primeira ingestão do registro (não muda em re-execuções graças ao UPSERT).

fato_votos

- Contém o posicionamento individual e nominal de cada parlamentar em uma votação específica.

Campo	Tipo	Restrição	Descrição
id	int4	PK / Identity	Chave primária sequencial

Campo	Tipo	Restrição	Descrição
			auto-incremental da tabela.
id_votacao	varchar	Non-Nullable	ID da votação correspondente (Relaciona-se com fato_votacoes).
tipo_voto	varchar	Nullable	O voto computado do deputado (ex: Sim, Não, Abstenção, Obstrução).
id_deputado	int4	Nullable	ID do parlamentar que votou (Relaciona-se com dim_deputados).
created_at	timestampz	Nullable	Data de inserção do registro de voto (carga incremental delete+insert escopada por <code>id_votacao</code>).

Estratégia de carga (Etapa 3 — `src/transformation.py`):

Tabela	Modo	Por quê
<code>dim_deputados</code> , <code>dim_partidos</code>	TRUNCATE + reload	Catálogos completos devolvidos do zero pela API em toda execução — não há histórico a preservar.
<code>fato_proposicoes</code> , <code>fato_votacoes</code>	UPSERT (INSERT ... ON CONFLICT (pk) DO UPDATE)	Registro novo → INSERT com <code>created_at = now()</code> . Registro existente → UPDATE dos campos da API sem tocar em <code>created_at</code> nem nas colunas de IA (<code>resumo_executivo</code> , <code>tema</code> ,

Tabela	Modo	Por quê
		<code>tema_score</code> , <code>data_tema</code>). A constraint UNIQUE/PK necessária para o <code>ON CONFLICT</code> é garantida de forma idempotente (<code>ALTER TABLE ... ADD CONSTRAINT ... UNIQUE</code>) caso ainda não exista.
<code>fato_proposicoes_adores</code> , <code>fato_votos</code>	DELETE + INSERT escopado (<code>carregar_incremental_assoc</code>)	Sem PK própria. Apaga só os registros associados aos <code>id_proposicao/id_votacao</code> do lote atual e reinsere — preserva associações de execuções anteriores.

Essa estratégia torna a carga **idempotente e cumulativa**: rodar o pipeline diariamente faz o banco crescer, e `created_at` reflete a data real da primeira ingestão — o que sustenta o filtro `created_at >= NOW() - INTERVAL '24h'` usado no digest do n8n.

Automação com n8n e evidências

Banco de dados (Supabase)

- **Projeto:** `bussola-publica` — Reference ID `thtdslaojkzbnzvbvfjvu`
- **URL:** <https://thtdslaojkzbnzvbvfjvu.supabase.co>

O Table Editor/SQL Editor do Supabase exige login (não há link público de visualização no plano Free). Como evidência de que o banco está populado com $\geq$ 30 dias de dados e a camada de IA está gravando `tema/resumo_executivo`, seguem os prints abaixo, extraídos do Table Editor e SQL Editor do projeto acima.

Workflow n8n

[Agendamento 06h] → [Rodar Pipeline (main.py)] → [Pipeline OK?]

└─ SIM → [Consultar Proposições do Dia] → [Montar Digest HTML] → [Enviar Digest]

└─ NÃO → [Alerta de Falha]

- Workflow exportado em [n8n/bussola_publica_ingestao_diaria.json](#).
- Cron `0 6 * * *` (06h, timezone `America/Sao_Paulo`).
- Consulta as proposições das últimas 24h já com `tema` e `resumo_executivo` e monta um digest HTML por e-mail.
- Ramo de alerta dedicado envia o `stderr` por e-mail caso o pipeline falhe.
- Passo a passo de importação e configuração de credenciais (Postgres/Supabase via Session Pooler + SMTP) em [n8n/GUIA_IMPORTACAO_n8n.md](#).

Evidências (prints)

Execução do workflow n8n (todos os nós verdes)	Digest diário por e-mail (tema + resumo IA)
	
<code>fato_proposicoes</code> no Supabase com <code>tema/resumo_executivo</code>	Cobertura de dados ≥ 30 dias (<code>data_apresentacao</code>)
	

Prompts e lógica utilizados na Camada de IA

Caminho B — Resumo Executivo (`src/ai_layer.py`)

Modelo: **gpt-4o-mini** (10x mais barato que o gpt-4o, qualidade adequada para resumos de 3 frases).

System prompt:

Você é um analista legislativo sênior da consultoria Bússola Pública.

Sua função é transformar ementas técnicas de proposições da Câmara dos Deputados em resumos claros e acionáveis para executivos e áreas de relações governamentais.

Regras para o resumo:

- Máximo 3 frases objetivas
- Linguagem direta, sem jargão jurídico
- Estrutura: (1) O que propõe, (2) Quem/o que é impactado, (3) Ponto de atenção para empresas
- Se a ementa for muito técnica ou vaga, informe isso claramente
- Responda APENAS com o resumo, sem introduções como 'O resumo é:' ou 'Esta proposição...'

User prompt (template):

Proposição: {sigla_tipo} {numero}/{ano}

Ementa oficial:

{ementa}

Gere o resumo executivo:

O resumo gerado é gravado em `fato_proposicoes.resumo_executivo` (+ `data_resumo`), e um backup local em JSON é salvo em `data/processed/`.

Caminho A — Classificação Temática por Embeddings (`src/classificacao_tematica.py`)

Modelo: **text-embedding-3-small** (~\$0.00002 / 1K tokens).

1. Gera o embedding da **ementa** de cada proposição pendente (**tema IS NULL**).
2. Gera, uma única vez por execução (com cache em memória), o embedding de cada um dos ~11 temas do catálogo de negócio — cada tema é descrito por uma frase rica em vocabulário, não só uma palavra:

Tema	Descrição (texto embedado)
Tecnologia e IA	Tecnologia, inteligência artificial, dados pessoais, internet, telecomunicações, inovação, startups, software, plataformas digitais e regulação de algoritmos.

Tema	Descrição (texto embedado)
Tributário	Tributos, impostos, reforma tributária, carga fiscal, ICMS, IRPF, isenções, incentivos fiscais e arrecadação.
Saúde	Saúde pública, SUS, medicamentos, planos de saúde, vigilância sanitária, hospitais, vacinas e profissionais de saúde.
Trabalho e Previdência	Direitos trabalhistas, CLT, emprego, salário mínimo, sindicatos, previdência social, aposentadoria e relações de trabalho.
Meio Ambiente	Meio ambiente, clima, licenciamento ambiental, desmatamento, saneamento, energia renovável, resíduos e sustentabilidade.
Economia e Finanças	Economia, mercado financeiro, bancos, crédito, juros, inflação, câmbio, investimentos e orçamento público.
Educação	Educação básica e superior, escolas, universidades, FIES, professores, currículo, financiamento educacional e ensino.
Segurança Pública	Segurança pública, polícia, crime, armas, código penal, sistema prisional e combate ao tráfico.
Agronegócio	Agronegócio, agricultura, pecuária, crédito rural, defensivos, exportação de commodities e produção no campo.
Infraestrutura e Transporte	Infraestrutura, rodovias, portos, aeroportos, mobilidade urbana, concessões, obras públicas e transporte.

Tema	Descrição (texto embedado)
Direitos e Cidadania	Direitos humanos, igualdade, direitos do consumidor, família, minorias, acesso à justiça e cidadania.

3. Calcula a **similaridade de cosseno** entre o embedding da ementa e o embedding de cada tema.
4. O tema de maior similaridade é a classificação; se o melhor score ficar abaixo do limiar **LIMIAR_TEMA** (padrão **0.20**), a proposição recebe o tema **"Outros"**.
5. Grava **tema**, **tema_score** e **data_tema** em **fato_proposicoes** e salva backup local em JSON.

Por que cosseno em vez de pedir o tema direto ao LLM?

- **Custo:** 1 embedding por ementa é ordens de magnitude mais barato que uma chamada de chat por proposição.
- **Consistência:** a lista de temas é fixa; um LLM generativo poderia inventar rótulos novos a cada execução. O cosseno sempre escolhe um tema do catálogo controlado.
- **Auditabilidade:** o score fica salvo em **tema_score**, dando transparência à classificação.



Como

Executar o Projeto:

Siga os passos abaixo para clonar o repositório, configurar o ambiente virtual com o Poetry, definir as variáveis de ambiente e executar o pipeline de inteligência legislativa.



****Pré-requisitos**:**

Antes de começar, certifique-se de ter instalado em sua máquina:

- **Python** (versão ^3.11 requisitada pelo projeto)
- **Poetry** (gerenciador de pacotes e ambientes virtuais)
- **Git**



****Passo a Passo**:**

1. **Clonar o Repositório e Acessar a Pasta:** Abra o seu terminal e execute os comandos abaixo para clonar o projeto e entrar no diretório raiz:

```
git clone
```

```
https://github.com/micaellimaj/Bussola-Publica-Pipeline-de-Inteligencia-Legislativa-com-IA.git
```

```
cd Bussola-Publica-Pipeline-de-Inteligencia-Legislativa-com-IA
```

2. **Instalar as Dependências com o Poetry:**

O projeto utiliza o Poetry para isolar o ambiente e gerenciar as bibliotecas estruturadas no pyproject.toml (como pandas, sqlalchemy, openai, entre outras). Instale todas as dependências executando:

```
poetry install
```

Este comando criará o ambiente virtual automaticamente e instalará os pacotes nas versões exatas necessárias.

3. Configurar as Variáveis de Ambiente (.env):

O pipeline precisa de credenciais do banco de dados e da API da OpenAI para funcionar.

- Duplica o arquivo de exemplo para criar o seu arquivo .env definitivo:

```
cp .env.example .env
```

- Abra o arquivo .env recém-criado no seu editor (como o VS Code) e preencha os campos com as suas credenciais reais conforme o modelo abaixo:

```
#
=====

# BUSSOLA PUBLICA - Variáveis de Ambiente

#
=====

# --- PostgreSQL (Supabase / Neon / Railway) ---

# No Supabase: Settings > Database > Connection string > URI

DATABASE_URL=postgresql://usuario:senha@host:5432/banco

# --- OpenAI API ---

# Obtenha em: https://platform.openai.com/api-keys

OPENAI_API_KEY=sk-proj-SUA_CHAVE_REAL_AQUI

# --- Configurações do Pipeline de IA (Etapas 4 e 5) ---

# DRY_RUN=true -> Modo Simulação: apenas estima custos de tokens, não consome API e
não grava no banco.

# DRY_RUN=false -> Modo Produção: executa o enriquecimento real e salva os dados.
```

DRY_RUN=true

Quantidade de proposições pendentes a processar por lote/execução

BATCH_SIZE=10

Modelo OpenAI escolhido (gpt-4o-mini é ~10x mais barato e ideal para os resumos)

MODELO_IA=gpt-4o-mini

--- Classificação Temática por Embeddings (Etapa 5 - Caminho A) ---

Modelo de embedding usado para calcular a similaridade de cosseno entre a

ementa da proposição e cada tema (Tecnologia, Tributário, Saúde, etc.)

MODELO_EMBEDDING=text-embedding-3-small

Limiar mínimo de similaridade de cosseno (0 a 1) para aceitar a classificação

de um tema. Abaixo desse valor, a proposição recebe o tema padrão "Outros".

LIMIAR_TEMA=0.20

4. Executar o Pipeline:

Com o ambiente configurado e as credenciais prontas, você pode rodar o pipeline através do Poetry:

- **Pipeline completo (extração + carga + IA):**

```
poetry run python main.py
```

- **Pipeline rápido (sem nova extração — carga + IA sobre o **data/raw** já existente):**

```
poetry run python main2.py
```

Ambos os pontos de entrada executam, em sequência: Etapa 3 (carga incremental no Supabase) → Etapa 4 (resumo executivo via GPT) → Etapa 5 (classificação temática via embeddings). Se o **DRY_RUN** estiver definido como **true**, você verá no console o diagnóstico de custos e o volume de proposições prontas para processamento, garantindo total controle financeiro antes de consumir os créditos da API.

Estrutura

de Pastas e Arquivos:

Abaixo está a arquitetura modular implementada no projeto para garantir a separação de responsabilidades em cada etapa do pipeline:

- |— .venv/ # Ambiente virtual local
- |— .vscode/ # Configurações do editor (settings.json)
- |— data_raw/ # Data Lake - Camada Bronze (Arquivos JSON brutos)
- | |— deputados/ # JSONs de deputados com timestamp
- | |— partidos/ # JSONs de partidos
- | |— proposicoes/ # JSONs de proposições e autores
- | └─ votacoes/ # JSONs de votações e votos
- |— docs/ # Documentações e relatórios das etapas
- | └─ Etapa4_Camada_IA.pdf # Relatório de especificação da camada de IA

- | — logs/ # Logs de execução do pipeline
 - | — transformacao_20260528.log # Registro histórico de transformações
- | — n8n/ # Etapa 5: automação
 - | — bussola_publica_ingestao_diaria.json # Workflow exportado (cron 06h + digest + alerta)
- | — GUIA_IMPORTACAO_n8n.md # Passo a passo de importação e configuração de credenciais
- | — readme/ # Imagens e prints usados neste README
 - | — legodadosbanner.png
 - | — roadmap.png
 - | — schema.png
- | — prints/ # Evidências de execução (n8n, digest, Supabase)
- | — src/ # Código-fonte principal do projeto
 - | — __pycache__/
 - | — _init_.py
 - | — ai_layer.py # Etapa 4: Integração com OpenAI (Resumo Executivo - Caminho B)
 - | — classificacao_tematica.py # Etapa 5: Classificação temática via embeddings (Caminho A)
 - | — config.py # Configurações globais e variáveis de ambiente
 - | — diagnostico.py # Script de validação e saúde do ambiente
 - | — extraction.py # Etapa 1: Scripts de extração/ingestão da API
 - | — transformation.py # Etapa 3: Classe PipelineEtapa3 (Orquestrador de carga)

└─ transformers.py	# Funções de transformação e limpeza com Pandas
└─ .env	# Variáveis de ambiente locais (Credenciais)
└─ .env.example	# Modelo de configuração das variáveis de ambiente
└─ .gitignore	# Arquivos ignorados pelo Git
└─ LICENSE	# Licença do projeto
└─ main.py	# Ponto de entrada do pipeline completo (extração + carga + IA)
└─ main2.py	# Ponto de entrada rápido (carga + IA, sem nova extração)
└─ poetry.lock	# Trava de versões das dependências
└─ pyproject.toml	# Configurações do projeto e dependências (Poetry)
└─ README.md	# Documentação do projeto

Decisões de escopo

- **Tabela fato *despesas* (cota parlamentar):** não implementada. O desafio cita *despesas* como exemplo de tabela fato, mas o produto da Bússola Pública foca em **proposições, votações e temas** — a cota parlamentar não alimenta o digest nem os alertas atuais. É uma decisão de escopo (não um item pendente) e pode ser adicionada em uma iteração futura sem impacto no modelo atual (bastaria uma nova tabela fato *despesas* ligada a *dim_deputados*, carregada com a mesma estratégia de UPSERT).
- **Alerta via Telegram:** não implementado. O desafio sugere e-mail OU Telegram para alertas; o workflow n8n já cobre o mesmo objetivo via **e-mail** — digest diário com as proposições de tema crítico + ramo dedicado de alerta em caso de falha do pipeline (*stderr*). Telegram ficaria redundante com a solução de e-mail entregue.

Segurança

- O arquivo *.env* (com *DATABASE_URL* e *OPENAI_API_KEY* reais) **não é versionado** — está no *.gitignore*. Use sempre o *.env.example* como modelo.
- O workflow do n8n usa **placeholders** de credencial (*REPLACE_POSTGRES_CRED_ID*, *REPLACE_SMTP_CRED_ID*); as credenciais reais ficam apenas na sua instância local do n8n.

About

Este repositório contém o Projeto Integrador da pós-graduação em Engenharia de Dados e Inteligência Artificial. O objetivo é desenvolver um pipeline de dados completo (ETL) que automatiza a captura, organização e análise de dados da API de Dados Abertos da Câmara dos Deputados.